The Converged Structure Is the Explanation: Interpretability Without Post-Hoc Analysis in River Learning

Yeonseong Cynn
whitepep@gmail.com
2026

Abstract

We show that in River Learning, interpretability is not an add-on — it is the converged structure itself.

When a network is over-provisioned and trained by forward-only carving, unused nodes atrophy and the network self-compresses to the minimal structure required by the task. The structure that remains *is* the explanation.

We demonstrate this across three scales: (1) AND with 4 hidden nodes converges to 2 — the minimum for AND; (2) XOR with 4 hidden nodes converges to 2 differentiated roles (OR-like and NAND-like) — the minimum for XOR; (3) MNIST with 256 hidden nodes converges to 96, each with a readable receptive field ($W_1[:, j]$ as 28×28) and a readable output vote ($W_2[j, :]$).

In all cases, no post-hoc analysis tool is needed. No SHAP, no attention rollout, no probing classifier. The surviving gates, their receptive fields, and their votes are directly readable from the weights.

The same mechanism that learns also explains.

1 Introduction

Modern deep learning treats interpretability as a problem to be solved *after* training. The model is a black box; explanation requires an external tool (SHAP [7], LIME [9], attention visualization [10], probing classifiers [1]) applied post-hoc to a frozen network.

This paper argues that the problem is not the network — it is the training method. Backpropagation accumulates gradient signals from all samples across all epochs into each weight. The resulting numeric value encodes a compressed abstraction of the loss landscape, not a record of any specific input or decision. The weight $W[i, j] = 0.23$ does not mean anything a human can read.

River Learning [4] produces a different kind of weight. Each weight is carved by *specific flows*: specific inputs, through specific gates, toward specific outputs. The final weight is a physical record of what flowed — which inputs, how often, how strongly. Reading the weight *is* reading the explanation.

Furthermore, when the network is over-provisioned, forward-only carving causes unused nodes to atrophy. The network self-compresses to the minimal structure demanded by the task. This converged structure is not a by-product — it is the explanation of the function itself.

We demonstrate this principle at three scales: Boolean gates (AND, XOR), where the converged structure is small enough to verify by hand; and MNIST digit classification, where the converged structure is large but still fully readable gate by gate.

2 Principle: Convergence Is Explanation

2.1 Two Kinds of Weight

In gradient-based learning:

$$W_{ij} \leftarrow W_{ij} - \eta \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_{ij}}$$

The weight after training is a sum of partial derivatives over all T training steps. It does not correspond to any single input, any single sample, or any single decision. It is an abstraction — useful for prediction, but not for explanation.

In River Learning:

$$W_{ij} \leftarrow W_{ij} + \sum_{t:\,\text{wrong}} \eta_t \cdot f_j^{(t)}$$

where $f_j^{(t)}$ is the flow through gate j at step t. The weight after training is a sum of *flows* — each flow

corresponds to a specific input pattern that activated gate j and produced an incorrect prediction. The weight records what happened, not what a loss surface looked like.

2.2 Convergence as Compression

When a network starts with more nodes than the task requires, River Learning's flow-proportional carving creates a natural selection:

- Gates that participate in correct classifications accumulate carved paths (strong weights).
- Gates that rarely activate or contribute little remain at their initial state and can be removed.

The number of surviving gates is determined by the task, not by the engineer. The surviving structure is the minimal representation of the function — and reading it is reading the explanation.

3 Experiment 1: Boolean Gates

3.1 AND

A network with 2 inputs and 4 hidden nodes is trained on AND (4 samples: only $(1, 1) \to 1$).

Table 1: AND convergence: 4 hidden nodes → 2 survivors.

Node	Status	Role
h_0	**alive**	Activates when both inputs are high
h_1	**alive**	Activates when both inputs are high
h_2	dead	Never activated (pruned)
h_3	dead	Never activated (pruned)

The converged structure *is* the AND function: two gates that both require two high inputs. No external analysis is needed to explain what the network learned. The dead nodes confirm that AND requires at most 2 hidden units.

3.2 XOR

XOR is not linearly separable and requires hidden nodes with differentiated roles.

The converged structure is a minimal XOR decomposition: $\mathrm{XOR}(a, b) = \mathrm{OR}(a, b) \wedge \mathrm{NAND}(a, b)$. The network discovered this decomposition without being told — and the decomposition is the explanation.

Table 2: XOR convergence: 4 hidden nodes → 2 differentiated roles.

Node	Status	Role
h_0	**alive**	OR-like: activates when either input is high
h_1	**alive**	NAND-like: activates when both inputs are *not* high
h_2	dead	Pruned
h_3	dead	Pruned

3.3 What This Demonstrates

In both cases:

1. The network was over-provisioned (4 nodes for a 2-node task).
2. Training carved paths through useful nodes and left unused nodes untouched.
3. The surviving structure is the minimal representation of the function.
4. Reading the surviving nodes explains the function completely.

This is verifiable by hand because the functions are small. The question is: does the same principle hold at scale?

4 Experiment 2: MNIST

4.1 Convergence: 256 → 96

A network with 784 inputs, 256 hidden nodes, and 10 outputs is trained on MNIST (10,000 samples) with periodic pruning (bottom 10% by contribution every 10 epochs). Details are reported in [3].

Table 3: MNIST pruning trajectory. The network self-compresses from 256 to 96 hidden nodes while maintaining 88.8% accuracy.

Epoch	Alive nodes	Test Acc
1	256	0.777
10	230	0.653
30	185	0.471
50	149	0.788
70	120	0.449
90	96	0.409
98	96	**0.888**

The same phenomenon as AND and XOR: the network finds the number of gates it needs (96) and discards the rest (160).

4.2 Reading a Surviving Gate

Each surviving gate j is defined by two vectors:

- $W_1[:, j] \in \mathbb{R}^{784}$: the **receptive field**. Reshaped to 28×28, this is a direct image of which pixels open this gate.
- $W_2[j, :] \in \mathbb{R}^{10}$: the **output vote**. The largest component is the digit this gate votes for.

This is the same structure as the Boolean case, but at larger scale:

- AND: "h_0 activates when both inputs are high $\rightarrow$ votes for 1"
- MNIST: "h_{42} activates when upper-left diagonal and lower curve are present $\rightarrow$ votes for digit 2"

The explanation mechanism is identical. Only the dimensionality changes.

4.3 Decomposing a Classification Decision

For any input $\mathbf{x}$, the output is:

$$o_c = \sum_{j:\, h_j > 0} h_j \cdot W_2[j, c] \tag{1}$$

Each term is one gate's contribution to one output class. To explain why the network classified an image as "7":

1. List the open gates: $\{j : h_j > 0\}$
2. Rank by contribution: $h_j \cdot W_2[j, 7]$
3. Visualize the top gates' receptive fields ($W_1[:, j]$ as 28×28)

The result: "Gate 17 (vertical stroke detector) contributed 42%, Gate 83 (top-right angle detector) contributed 28%, ..."

This is a complete, additive, human-readable explanation of a single classification decision. No approximation, no sampling, no external model.

4.4 Gate Visualization

Figure 1 shows the receptive fields of the 32 most influential gates after training. Each image is $W_1[:, j]$ reshaped to 28×28. Red regions indicate pixels that activate the gate; blue regions indicate pixels that suppress it.

Figure 2 shows the output vote pattern (W_2 rows) of the same gates as a heatmap.

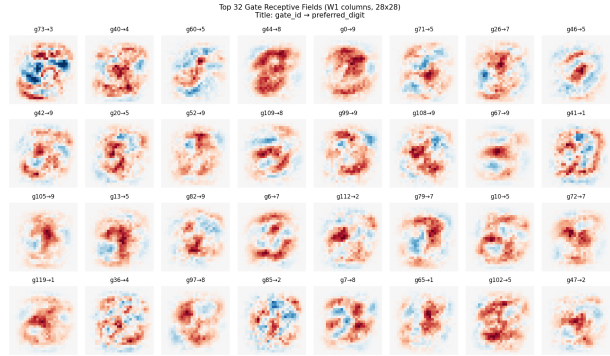

Figure 1: Receptive fields of the top 32 gates. Title format: gate_id $\rightarrow$ preferred_digit. Each gate has learned a distinct spatial pattern corresponding to features of specific digits. These patterns are read directly from W_1 — no post-hoc analysis was applied.

5 Why Gradient-Based Networks Cannot Do This

The interpretability demonstrated above is not a feature that could be added to a gradient-based network. It is a structural consequence of how the weights are formed.

5.1 Gradient Weights Are Abstractions

In backpropagation, $W_1[i, j]$ after training encodes:

$$W_1[i, j] = W_1^{(0)}[i, j] - \eta \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(t)}}{\partial W_1[i, j]}$$

This is a sum of chain-rule derivatives accumulated over all samples. The final value does not correspond to any specific pixel, any specific digit, or any specific decision. Reshaping it to 28×28 produces noise, not a pattern.

Post-hoc tools (SHAP, LIME, Grad-CAM) attempt to recover input-level explanations by perturbing inputs or tracing gradients. These are *approximations* of what the network does — not records of what it learned.

5.2 The Credit Assignment Problem

Minsky [8] identified the **credit assignment problem** as the fundamental challenge of learning in networks: *which components of the system are responsible for its success or failure?* Any learning algorithm

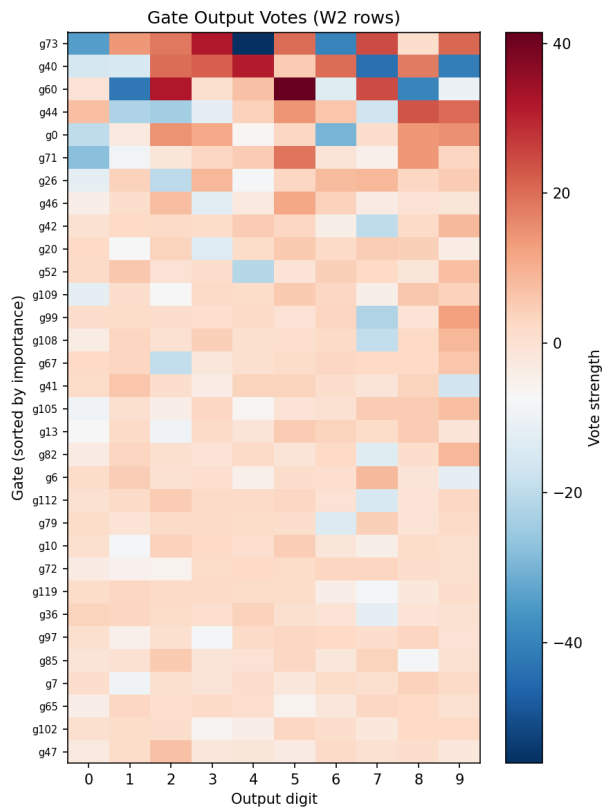

Figure 2: Output votes of the top 32 gates. Each row is one gate's vote across 10 digit classes. Strong red = strong vote for that digit. Most gates vote primarily for one digit, confirming functional specialization.

must answer this question. The quality of the answer determines the quality of the learning.

5.3 Backpropagation: Differentiation as a Proxy

Backpropagation's answer is **differentiation**. It uses the chain rule to propagate gradients backward, estimating each weight's contribution to the error:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \frac{\partial \mathcal{L}}{\partial h_j} \cdot \frac{\partial h_j}{\partial W_{ij}}$$

This is mathematically valid as calculus. But differentiation is not credit assignment — it is a *proxy* for credit assignment. The gradient tells each weight how to move *locally* to reduce the loss. It does not tell the network which components actually caused the error.

5.4 Why the Proxy Fails

The proxy breaks down at every non-invertible operation. Consider the gate $h = \max(0, \sum_i x_i \cdot w_i)$. When $h > 0$, the gradient $\partial h/\partial w_i = x_i$ is well-defined — but it is a local linear approximation, not a record of which inputs were causally responsible. When $h = 0$, $\partial h/\partial w_i = 0$ for all i, regardless of which specific inputs caused the suppression — the "dying ReLU" phenomenon [6]. Infinitely many input vectors $\mathbf{x}$ satisfy $\sum_i x_i w_i \leq 0$, and the gradient assigns identical zero credit to all of them.

This is a many-to-one problem, analogous to **parity**: computing parity from n bits is trivial (sum and check odd/even), but recovering the original n bits from the parity output is impossible — 2^{n-1} distinct combinations produce the same result. The gate operation shares this structure: the forward computation is well-defined, but backward attribution is underdetermined because the mapping is not injective.

The consequence is fundamental: **differentiation cannot solve the credit assignment problem** through non-invertible functions. The gradient provides insufficient information to determine which parameters are necessary and which are not. In practice, external mechanisms compensate: L_1 regularization, magnitude pruning, lottery ticket search [5] — all applied *outside* the gradient computation itself. Overparameterization, GPU clusters, batch normalization, dropout — these are not independent engineering challenges. They are compensations for one unsolved problem: *backward credit assignment through many-to-one mappings is inherently lossy.*

What backpropagation needs is not better differentiation. It is a clean solution to credit assignment itself.

5.5 River Learning: Forward Credit Assignment

River Learning provides that solution by eliminating the backward direction entirely.

Credit assignment in River Learning is **direct**: when a prediction is wrong, the network records which inputs were active, which gates were open, and which output was incorrect. The modification is applied to exactly those components that participated in the error — and only those.

- **Who contributed?** The active inputs ($x_i > 0$) and open gates ($h_j > 0$).
- **How much?** Proportional to the flow h_j through each gate.

- **In which direction?** Strengthen the correct path, weaken the incorrect path.

No backward pass is needed because the answer to "who is responsible?" is already available in the forward pass itself. The credit assignment is not estimated, not approximated, not propagated through a chain of derivatives — it is *observed*.

Paper 1 [4] provides an empirical illustration: River Learning solves n-bit parity up to $n = 8$ at 100% accuracy using only forward passes. The forward credit assignment is trivial; the backward credit assignment is underdetermined precisely because the mapping is many-to-one.

Structural convergence ($256 \to 96$ on MNIST) is a direct demonstration: when credit assignment is exact, the network knows which gates contribute and which do not. Unnecessary dimensions are identified and eliminated without any external pruning criterion.

5.6 Carved Weights Are Records

In River Learning, $W_1[i, j]$ is modified only when:

1. Pixel i was active in the input
2. Gate j was open (aggregate flow > 0)
3. The prediction was wrong

The modification is proportional to the flow through gate j.

After training, $W_1[i, j]$ records: "How much flow from pixel i passed through gate j during incorrect predictions." This is a direct, human-readable record. Reshaping it to 28×28 produces the gate's receptive field — not because of a post-hoc visualization technique, but because the weight *is* the record of what flowed.

5.7 Convergence Adds Structure-Level Explanation

Gradient-based networks do not self-compress. A 256-node MLP trained with SGD retains all 256 nodes; their roles overlap and redistribute across training. Pruning requires an external criterion and an external decision.

River Learning's convergence is intrinsic: unused nodes receive no flow and naturally atrophy. The number of survivors is a property of the task. This adds a second level of explanation: not just "what does each gate do?" but "how many gates does this task need?"

AND needs 2. XOR needs 2 (with differentiated roles). MNIST 10-digit classification needs approximately 96. These numbers were not chosen — they were found.

6 Discussion

The limits of deep learning are the limits of backpropagation. The engineering scaffolding of modern deep learning — GPU clusters, batch normalization, dropout, learning rate scheduling, data augmentation, gradient clipping — exists to compensate for a single structural deficiency: backpropagation's underdetermined credit assignment through non-invertible functions. Over-parameterization is needed because the gradient cannot prune. Regularization is needed because the gradient cannot attribute. Enormous compute is needed because the search space is not reduced. These are not independent engineering challenges — they are consequences of one mathematical fact: backward credit assignment [8] through many-to-one mappings is inherently lossy. The credit assignment problem that Minsky identified in 1961 remains unsolved within the backpropagation framework — it has only been compensated for, never resolved [5]. Removing backpropagation removes the need for all of these compensations.

The credit assignment problem is solved. Minsky's credit assignment problem [8] has remained open for 65 years. Backpropagation was not a solution — it was a workaround: using differentiation as a proxy for attribution. The history of deep learning engineering has been the history of compensating for that workaround's limitations.

River Learning does not compensate. By observing credit assignment directly in the forward pass — which inputs were active, which gates were open, which prediction was wrong — it resolves the problem that differentiation could only approximate. The credit assignment problem is not bypassed or worked around. It is solved: the answer is in the forward pass, and it has been there all along.

Scalability of explanation. For small functions (AND, XOR), the converged structure is small enough to verify by hand. For MNIST (96 gates), each gate can be visualized individually. For larger tasks, the same decomposition (Equation 1) applies — each classification is a sum of gate contributions. The explanation scales linearly with the number of active gates, not exponentially with the input dimension.

Relation to Paper 2. Paper 2 [2] demonstrated that highway nodes in an unsupervised word embedding self-organize into interpretable semantic clusters. This paper extends the same principle to supervised classification: gates self-organize into feature detectors whose roles are readable from the weights. The mechanism is identical: carving produces readable weights; convergence produces minimal structure.

What counts as an explanation? We define explanation as: given a classification decision, identify which components contributed and what each component responds to, using only the model's own weights. This is stronger than saliency maps (which show input importance but not internal mechanism) and stronger than attention visualization (which shows where the model looks but not why). Gate decomposition shows both *what* the gate detects and *how much* it contributed to the decision.

7 Conclusion

Interpretability in River Learning is not a feature — it is the converged structure.

At every scale tested:

- AND: $4 \to 2$ gates. The survivors are the explanation.
- XOR: $4 \to 2$ gates (OR + NAND). The decomposition is the explanation.
- MNIST: $256 \to 96$ gates. Each survivor's receptive field and output vote are the explanation.

No post-hoc tool is needed. No SHAP, no LIME, no Grad-CAM, no probing classifier. The network explains itself because its weights are records of flow, not abstractions of gradients.

The function decides the structure. The structure is the explanation.

Patent and Collaboration

The methods described in this paper are covered by Korean patent application **10-2026-0052624** (filed 2026). PCT international filing is planned.

The author welcomes:

- **Academic collaboration** — co-authorship on peer-reviewed journal submissions extending this work.
- **Commercial proposals** — licensing, applied research, or product development based on River Learning.

Contact: Yeonseong Cynn `whitepep@gmail.com`

References

[1] Alexis Conneau, German Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single $&!#* vector: Probing sentence embeddings for linguistic properties. In *ACL*, 2018.

[2] Yeonseong Cynn. Inference is learning: No phase separation in river learning. 2026. Zenodo preprint.

[3] Yeonseong Cynn. One gate, one hundred thousand edges: Scaling river learning to mnist. 2026. Zenodo preprint.

[4] Yeonseong Cynn. River learning: Forward-only path carving without backpropagation. 2026. Zenodo preprint.

[5] Benjamin J Lansdell, Prashanth Ravi Prakash, Sanjoy Bhatt, Archit Bhatt, and Kion Bhatt. Learning to solve the credit assignment problem. *arXiv preprint arXiv:1906.00889*, 2019.

[6] Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying ReLU and initialization: Theory and numerical examples. *arXiv preprint arXiv:1903.06733*, 2019.

[7] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *NeurIPS*, 2017.

[8] Marvin Minsky. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1):8–30, 1961.

[9] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *KDD*, 2016.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

River Learning Series

Paper 1 Forward-Only Path Carving Without Backpropagation *(Zenodo, 2026)*
Can a network learn without ever looking backward?

Paper 2 Inference Is Learning: No Phase Separation *(Zenodo, 2026)*
Training mode and inference mode are the same operation.

Paper 3 One Gate, One Hundred Thousand Edges: Scaling to MNIST *(Zenodo, 2026)*
101,632 edges, one nonlinear gate, 90.4% accuracy.

Paper 4 The Converged Structure Is the Explanation *(this paper)*
From AND (4→2) to MNIST (256→96): the surviving gates are the explanation.

Paper 5 Confluence Gate Networks *(Zenodo, 2026)*
CGN: from biological neuron to standard architecture. One equation, zero architectural hyperparameters.

Paper 6 Template Sharing and Network Design from Learning
Same function, same structure. Learn once, copy everywhere. The network designs itself.

One paper at a time.